1, Install nginx and run nginx on port number 81.

Prerequisites

- AWS EC2 instance running Amazon Linux 2023.
- SSH access to the EC2 instance.
- Permission to modify AWS security group inbound rules.
- Internet connectivity to download packages.

Installation and Configuration Steps

Step 1: Connect to the EC2 Instance

Connect to the EC2 instance using SSH or Git Bash. Check the operating system:

cat /etc/os-release

Step 2: Install Nginx

Install Nginx using the DNF package manager:

```
sudo dnf install nginx -y
```

Step 3: Verify Nginx Installation

Check the installed version:

```
nginx -v
```

Step 4: Back Up the Configuration File

Create a backup of the existing Nginx configuration:

```
sudo cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.bak
```

Step 5: Configure Nginx to Use Port 81

Open the configuration file:

```
sudo nano /etc/nginx/nginx.conf
```

Inside the `http` section, find the default `server` block and change the listening port from `80` to `81`.

Example:

```
server {
    listen 81;
    server_name _;
    root /usr/share/nginx/html;
    index index.html index.htm;
}
```

If the configuration contains `listen [::]:80;`, change it to `listen [::]:81;` as well.

Save the file by pressing `Ctrl + O`, then `Enter`, and exit using `Ctrl + X`.

Step 6: Test the Configuration

Run:

```
sudo nginx -t
```

The expected output indicates that the syntax is OK and the configuration test is successful.

Step 7: Start and Enable Nginx

Start Nginx and enable it to start automatically after a reboot:

```
sudo systemctl enable --now nginx
```

Check the service status:

```
sudo systemctl status nginx --no-pager
```

Expected result: `Active: active (running).`

Step 8: Verify Port 81

Check whether Nginx is listening on port 81:

```
sudo ss -ltnp | grep ':81'
```

Test the server locally:

```
curl -I http://localhost:81
```

An HTTP response confirms that the server is responding locally.

Step 9: Configure AWS Security Group

1. Open the AWS Management Console.

2. Navigate to EC2 and select your instance.
3. Open the Security tab and select its security group.
4. Click Edit inbound rules.
5. Add the following rule:
    - Type: Custom TCP
    - Protocol: TCP
    - Port range: `81`
    - Source: My IP
6. Save the rules.

Use a broader source only if public access is required.

Step 10: Access Nginx in a Browser

Open the following URL, replacing the placeholder with your EC2 public IPv4 address:

```
http://<EC2-PUBLIC-IP>:81
```

The Nginx welcome page or configured website should appear.

Troubleshooting

Configuration error: Run `sudo nginx -t` and correct the reported error.

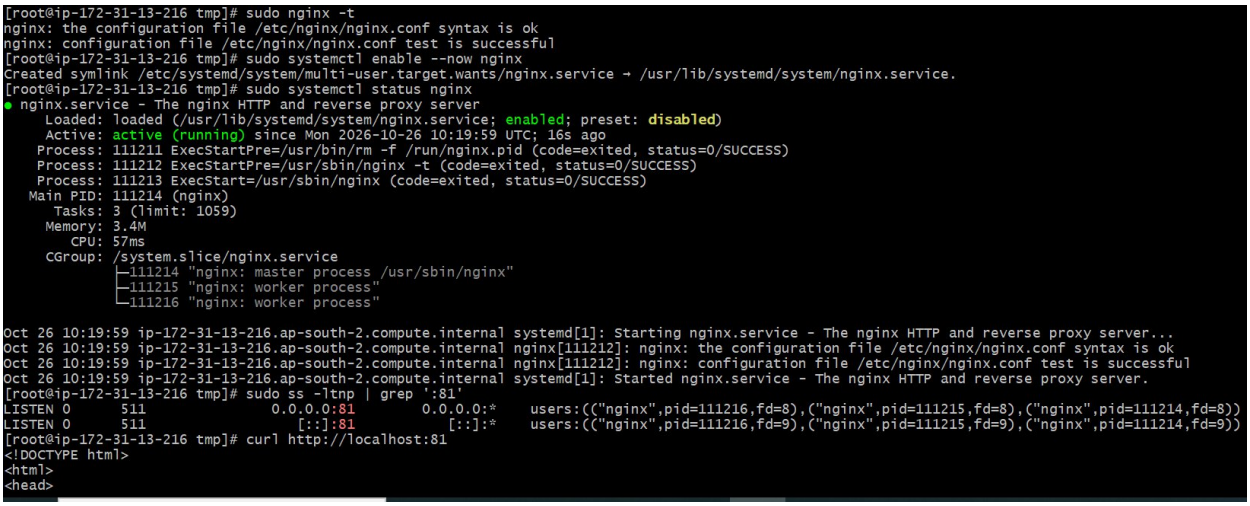
Port 81 is not listening: Check `/etc/nginx/nginx.conf`, test the configuration, and restart Nginx.

```
sudo nginx -t
sudo systemctl restart nginx
```

Browser connection timeout: Verify the AWS security group allows inbound TCP port 81 and Nginx is listening on that port.

Port already in use: Identify the process using port 81:

```
sudo ss -ltnp | grep ':81'
```



2, Deploy a sample index.html file on nginx.

Step 1: Create the `index.html` file

Open your terminal and run:

```
sudo nano /usr/share/nginx/html/index.html
```

Step 2: Add sample HTML code

Copy and paste the following code into the file:

```
<!DOCTYPE html>
<html>
<head>
    <title>My Nginx Website</title>
</head>
<body>
    <h1>Welcome to My Nginx Server!</h1>
    <p>This is my first website deployed on Nginx.</p>
    <p>Server: AWS EC2</p>
    <p>Port: 81</p>
</body>
</html>
```

Save the file:

1. Press `Ctrl + O`.
2. Press `Enter`.
3. Press `Ctrl + X`.

Step 3: Check the Nginx configuration

Run:

```
sudo nginx -t
```

Expected output:

```
syntax is ok
test is successful
```

Step 4: Restart Nginx

```
sudo systemctl restart nginx
```

Check the service:

```
sudo systemctl status nginx
```

Make sure the status shows `active (running)`.

Step 5: Test the website

Run this command on your EC2 terminal:

```
curl http://localhost:81
```

You should see the HTML content you created.

Step 6: Open the website in your browser

Enter this URL, replacing the example IP with your EC2 instance's public IPv4 address:

```
http://YOUR-EC2-PUBLIC-IP:81
```

If the website does not open, check your EC2 Security Group → Inbound rules and make sure Custom TCP, port 81 is allowed from your IP address.

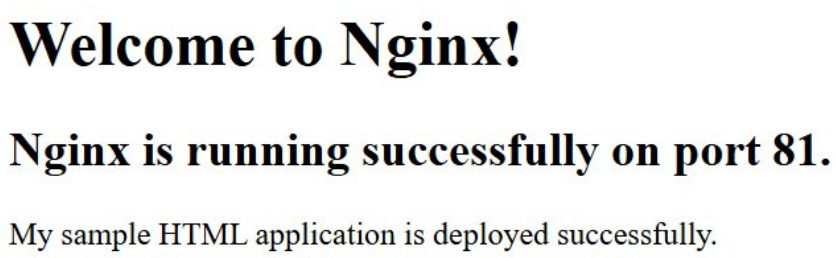
Step 7: Evidence for your assignment

Take screenshots of:

1. The `index.html` file and its HTML code.
2. The output of `sudo nginx -t`.
3. The output of `sudo systemctl status nginx`.

4. The website opened in your browser on port `81`.

Result: A sample `index.html` page has been deployed on Nginx and is accessible through port 81.



3, Install Apache and run Apache on port number 82

These steps are for your Amazon Linux EC2 instance.

Step 1: Install Apache

Run the following commands in your EC2 terminal:

```
sudo dnf update -y
sudo dnf install httpd -y
```

Check the Apache version:

```
httpd -v
```

Step 2: Configure Apache to use port 82

Open the Apache configuration file:

```
sudo nano /etc/httpd/conf/httpd.conf
```

Find this line:

```
Listen 80
```

Change it to:

```
Listen 82
```

If you cannot find the line, add `Listen 82` to the configuration file, ensuring there is no conflicting `Listen 80` directive.

Save and exit Nano:

1. Press `Ctrl + O`.
2. Press `Enter`.
3. Press `Ctrl + X`.

Step 3: Create a sample HTML page

Run:

```
sudo nano /var/www/html/index.html
```

Add this sample content:

```
<!DOCTYPE html>
<html>
<head>
    <title>Apache Web Server</title>
</head>
<body>
    <h1>Welcome to Apache Server!</h1>
    <p>Apache is running successfully on port 82.</p>
</body>
</html>
```

Save and exit using `Ctrl + O`, `Enter`, and `Ctrl + X`.

Step 4: Test the Apache configuration

Run:

```
sudo apachectl configtest
```

Expected output:

```
Syntax OK
```

If an error appears, correct it before starting Apache.

Step 5: Start Apache

Enable Apache to start automatically after a reboot, and start it now:

```
sudo systemctl enable --now httpd
```

Check the service:

```
sudo systemctl status httpd
```

Look for:

```
Active: active (running)
```

Step 6: Verify port 82

Run:

```
sudo ss -ltnp | grep ':82'
```

You should see Apache (`httpd`) listening on port `82`.

Test the webpage locally:

```
curl http://localhost:82
```

The output should contain your HTML content.

Step 7: Allow port 82 in AWS

1. Open the AWS Management Console.
2. Go to EC2 → Instances.
3. Select your instance.
4. Open its Security tab and click the associated security group.
5. Select Inbound rules → Edit inbound rules.
6. Add the following rule:

Setting	Value
Type	Custom TCP
Port range	`82`
Source	My IP

7. Save the rules.

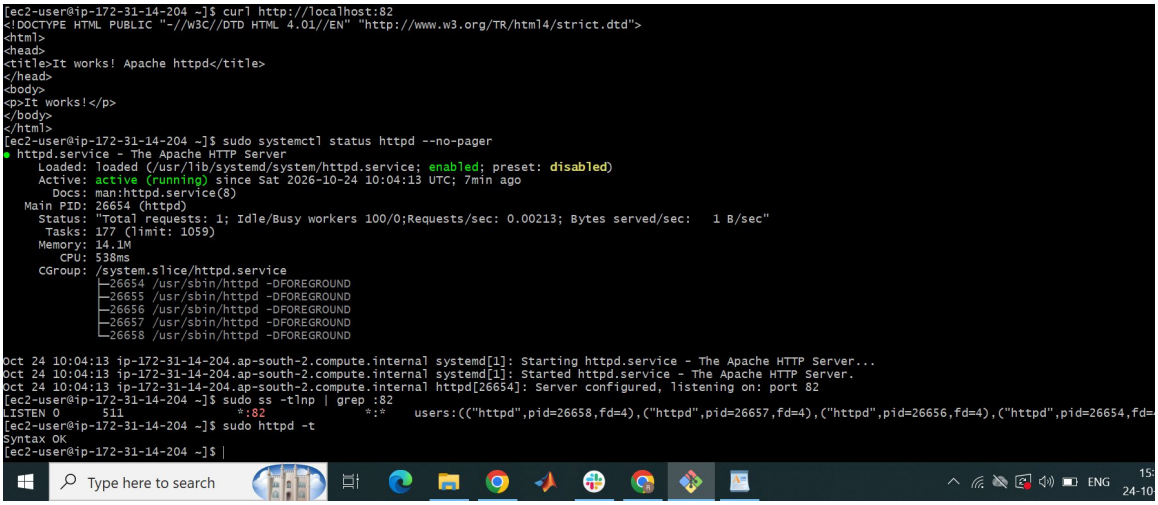
Step 8: Open Apache in your browser

Enter this address, replacing the placeholder with your EC2 public IPv4 address:

`http://YOUR-EC2-PUBLIC-IP:82`

You should see:

Welcome to Apache Server!





4, Deploy a sample index.html file on Apache.

Step 1: Create the HTML File

Open the terminal and run:

```
sudo nano /var/www/html/index.html
```

Step 2: Add Sample HTML Code

Copy and paste the following code into the file:

```
<!DOCTYPE html>
<html>
<head>
    <title>My Apache Website</title>
</head>
<body>
    <h1>Welcome to My Apache Server!</h1>
    <p>This is my first webpage deployed on Apache.</p>
```

```
    <p>Apache is running on port 82.</p>
</body>
</html>
```

Save the file by pressing `Ctrl + O`, then `Enter`, and exit using `Ctrl + X`.

Step 3: Restart Apache

Run the following command:

```
sudo systemctl restart httpd
```

Check the Apache service status:

```
sudo systemctl status httpd
```

Ensure the service shows `active (running)`.

Step 4: Verify the HTML Page

Run the following command:

```
curl http://localhost:82
```

The output should contain the HTML content you added.

Step 5: Access the Website in a Browser

Open your web browser and enter the following URL, replacing the placeholder with your EC2 public IPv4 address:

```
http://YOUR-EC2-PUBLIC-IP:82
```

You should see the message:

Welcome to My Apache Server!

If the webpage does not open, verify that port 82 is allowed in the EC2 security group's inbound rules and that Apache is listening on port 82.

```
[ec2-user@ip-172-31-14-204 ~]$ sudo nano /var/www/html/index.html
[ec2-user@ip-172-31-14-204 ~]$ sudo systemctl restart httpd
[ec2-user@ip-172-31-14-204 ~]$ sudo systemctl status httpd --no-pager
● httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
     Active: active (running) since Sat 2026-10-24 10:30:13 UTC; 12s ago
       Docs: man:httpd.service(8)
   Main PID: 28336 (httpd)
     Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec:   0 B/sec"
      Tasks: 177 (limit: 1059)
     Memory: 13.7M
        CPU: 68ms
     CGroup: /system.slice/httpd.service
             ├─28336 /usr/sbin/httpd -DFOREGROUND
             ├─28337 /usr/sbin/httpd -DFOREGROUND
             ├─28338 /usr/sbin/httpd -DFOREGROUND
             ├─28339 /usr/sbin/httpd -DFOREGROUND
             └─28401 /usr/sbin/httpd -DFOREGROUND

Oct 24 10:30:13 ip-172-31-14-204.ap-south-2.compute.internal systemd[1]: Starting httpd.service - The Apache HTTP Serve
Oct 24 10:30:13 ip-172-31-14-204.ap-south-2.compute.internal systemd[1]: Started httpd.service - The Apache HTTP Server
Oct 24 10:30:13 ip-172-31-14-204.ap-south-2.compute.internal httpd[28336]: Server configured, listening on: port 82
[ec2-user@ip-172-31-14-204 ~]$ curl http://localhost:82
<!DOCTYPE html>
<html>
<head>
    <title>My Apache Web Server</title>
</head>
<body>
    <h1>Welcome to My Apache Server!</h1>
    <h2>DevOps Assignment</h2>
    <p>Apache is running successfully on port 82.</p>
    <p>Deployed on AWS EC2 using Amazon Linux.</p>
</body>
</html>
[ec2-user@ip-172-31-14-204 ~]$
```

Welcome to My Apache Server!

DevOps Assignment

Apache is running successfully on port 82.

Deployed on AWS EC2 using Amazon Linux.

5, Install Apache tomcat on port number 8082

Step 1: Install Java

Apache Tomcat requires Java to run.

```
sudo dnf update -y
sudo dnf install java-17-amazon-corretto -y
```

Verify the Java installation:

```
java -version
```

Step 2: Download Apache Tomcat

Download the Tomcat 10.1.61 archive:

```
cd /tmp
wget https://archive.apache.org/dist/tomcat/tomcat-10/v10.1.61/bin/apache-tomcat-10.1.61.tar.gz
```

Extract the downloaded archive:

```
sudo tar -xzf apache-tomcat-10.1.61.tar.gz -C /opt
```

Rename the extracted directory to `tomcat`:

```
sudo mv /opt/apache-tomcat-10.1.61 /opt/tomcat
```

Check the directory:

```
ls /opt/tomcat
```

You should see directories such as `bin`, `conf`, `logs`, `webapps`, and `lib`.

Step 3: Configure Tomcat to Run on Port 8082

Open the Tomcat configuration file:

```
sudo nano /opt/tomcat/conf/server.xml
```

Find the HTTP Connector line, which normally contains:

```
<Connector port="8080" protocol="HTTP/1.1"
```

Change the port from `8080` to `8082`:

```
<Connector port="8082" protocol="HTTP/1.1"
```

Do not change the other Connector settings. Save the file using `Ctrl + O`, press `Enter`, and exit using `Ctrl + X`.

Important: If the file is empty or shows an XML error, do not save it as an empty file. Tomcat requires a valid `server.xml` configuration.

Step 4: Start Tomcat

Start Tomcat using its startup script:

```
sudo /opt/tomcat/bin/startup.sh
```

Check whether Java is listening on port 8082:

```
sudo ss -ltnp | grep ':8082'
```

If you have already created a `tomcat.service` systemd service, use that service instead:

```
sudo systemctl restart tomcat
sudo systemctl status tomcat
```

Use the startup script or the systemd service, not both independently.

Step 5: Verify Tomcat Locally

Run:

```
curl http://localhost:8082
```

If Tomcat starts successfully, you should receive the Tomcat welcome-page HTML.

You can also check the Tomcat logs:

```
sudo tail -n 50 /opt/tomcat/logs/catalina.out
```

Step 6: Allow Port 8082 in AWS

1. Open the AWS Management Console.
2. Navigate to EC2 → Instances.
3. Select your EC2 instance.
4. Open the Security tab and click the associated security group.
5. Select Edit inbound rules → Add rule.
6. Configure the rule:

Setting	Value

Type	Custom TCP
Port range	`8082`
Source	My IP

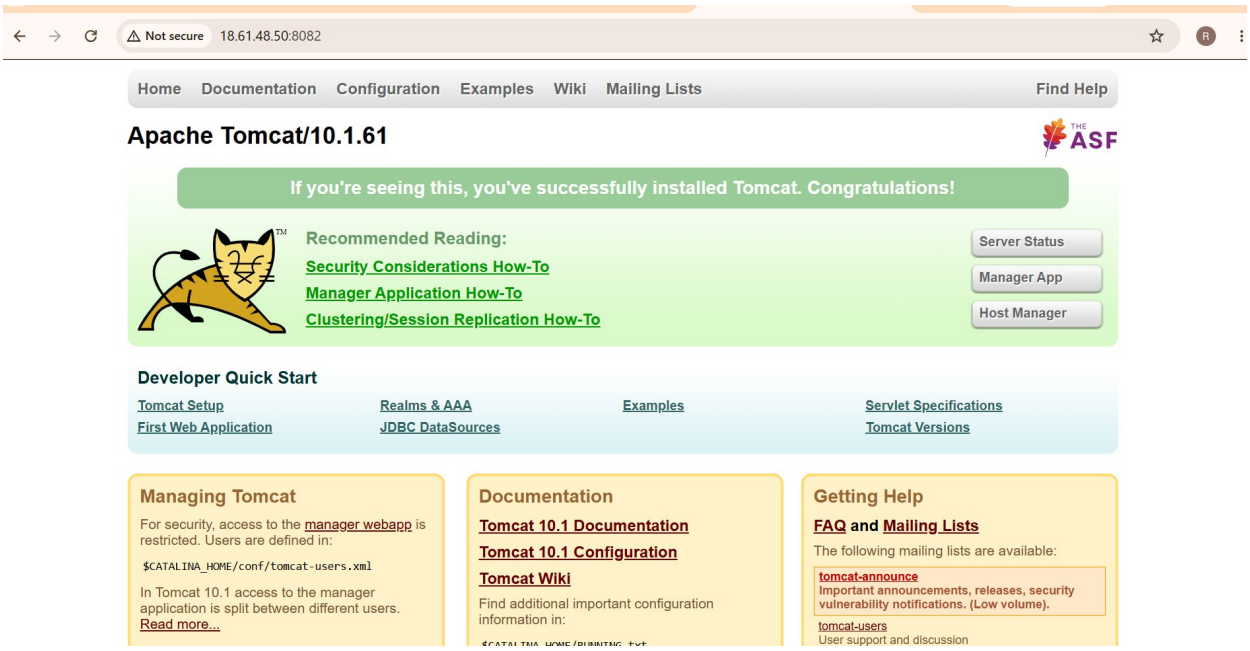
7. Save the inbound rules.

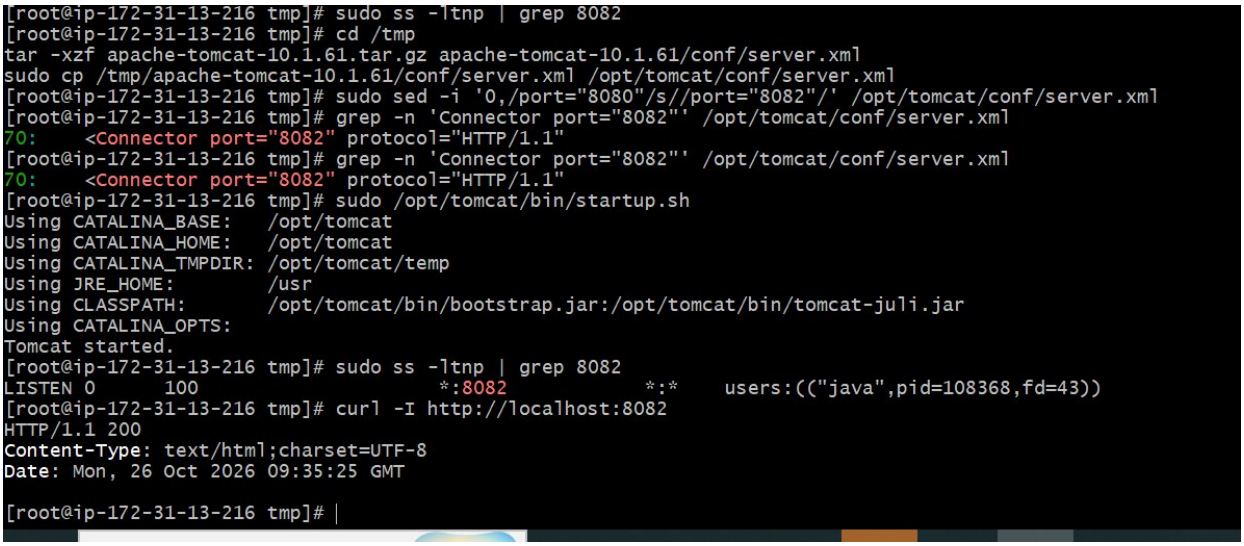
Step 7: Access Tomcat in a Browser

Open your browser and enter the following URL, replacing the placeholder with your EC2 public IPv4 address:

```
http://YOUR-EC2-PUBLIC-IP:8082
```

You should see the Apache Tomcat welcome page.





6, Deploy a sample app on webapps

Step 1: Create a Sample Application Directory

Run the following command on your EC2 terminal:

```
sudo mkdir -p /opt/tomcat/webapps/sample
```

Step 2: Create the `index.html` File

Open a new HTML file:

```
sudo nano /opt/tomcat/webapps/sample/index.html
```

Add the following code:

```
<!DOCTYPE html>
<html>
<head>
```

```
    <title>Sample Tomcat Application</title>
</head>
<body>
    <h1>Welcome to My Sample Application!</h1>
    <p>This application is deployed on Apache Tomcat.</p>
    <p>Tomcat is running on port 8082.</p>
</body>
</html>
```

Save the file by pressing `Ctrl + O`, then `Enter`, and exit using `Ctrl + X`.

Step 3: Set Permissions

Run:

```
sudo chmod -R 755 /opt/tomcat/webapps/sample
```

Step 4: Check Tomcat Status

If you started Tomcat using its startup script, run:

```
sudo /opt/tomcat/bin/startup.sh
```

If Tomcat is already running, you do not need to start it again.

Check whether port 8082 is listening:

```
sudo ss -ltnp | grep ':8082'
```

Step 5: Test the Application

Run this command:

```
curl http://localhost:8082/sample/
```

You should see the HTML content containing:

```
Welcome to My Sample Application!
```

Step 6: Open the Application in a Browser

Enter the following URL, replacing the placeholder with your EC2 public IPv4 address:

```
http://YOUR-EC2-PUBLIC-IP:8082/sample/
```

The browser should display your sample application.

Make sure port `8082` is allowed in your EC2 security group's inbound rules.



7, Create a tomcat.service file for tomcat.

Step 1: Create the Service File

Run the following command:

```
sudo nano /etc/systemd/system/tomcat.service
```

Step 2: Add the Service Configuration

Copy and paste the following content into the file:

```
[Unit]
Description=Apache Tomcat Web Application Server
After=network.target

[Service]
Type=forking

Environment="JAVA_HOME=/usr/lib/jvm/java-17-amazon-corretto"
Environment="CATALINA_HOME=/opt/tomcat"
Environment="CATALINA_BASE=/opt/tomcat"

ExecStart=/opt/tomcat/bin/startup.sh
ExecStop=/opt/tomcat/bin/shutdown.sh

Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Save and exit Nano:

1. Press `Ctrl + O`.
2. Press `Enter`.
3. Press `Ctrl + X`.

Step 3: Reload the systemd Daemon

Run:

```
sudo systemctl daemon-reload
```

This command makes systemd recognize the new service file.

Step 4: Enable the Tomcat Service

Run:

```
sudo systemctl enable tomcat
```

This enables Tomcat to start automatically when the system boots.

Step 5: Start Tomcat

Run:

```
sudo systemctl start tomcat
```

Step 6: Check the Service Status

Run:

```
sudo systemctl status tomcat
```

Expected status:

```
Active: active (running)
```

Step 7: Verify Port 8082

Check whether Tomcat is listening on port 8082:

```
sudo ss -ltnp | grep ':8082'
```

Test the Tomcat server:

```
curl http://localhost:8082
```

If you deployed the sample application earlier, test it using:

```
curl http://localhost:8082/sample/
```

Step 8: Manage Tomcat Using systemctl

You can now control Tomcat using these commands:

Start Tomcat:

```
sudo systemctl start tomcat
```

Stop Tomcat:

```
sudo systemctl stop tomcat
```

Restart Tomcat:

```
sudo systemctl restart tomcat
```

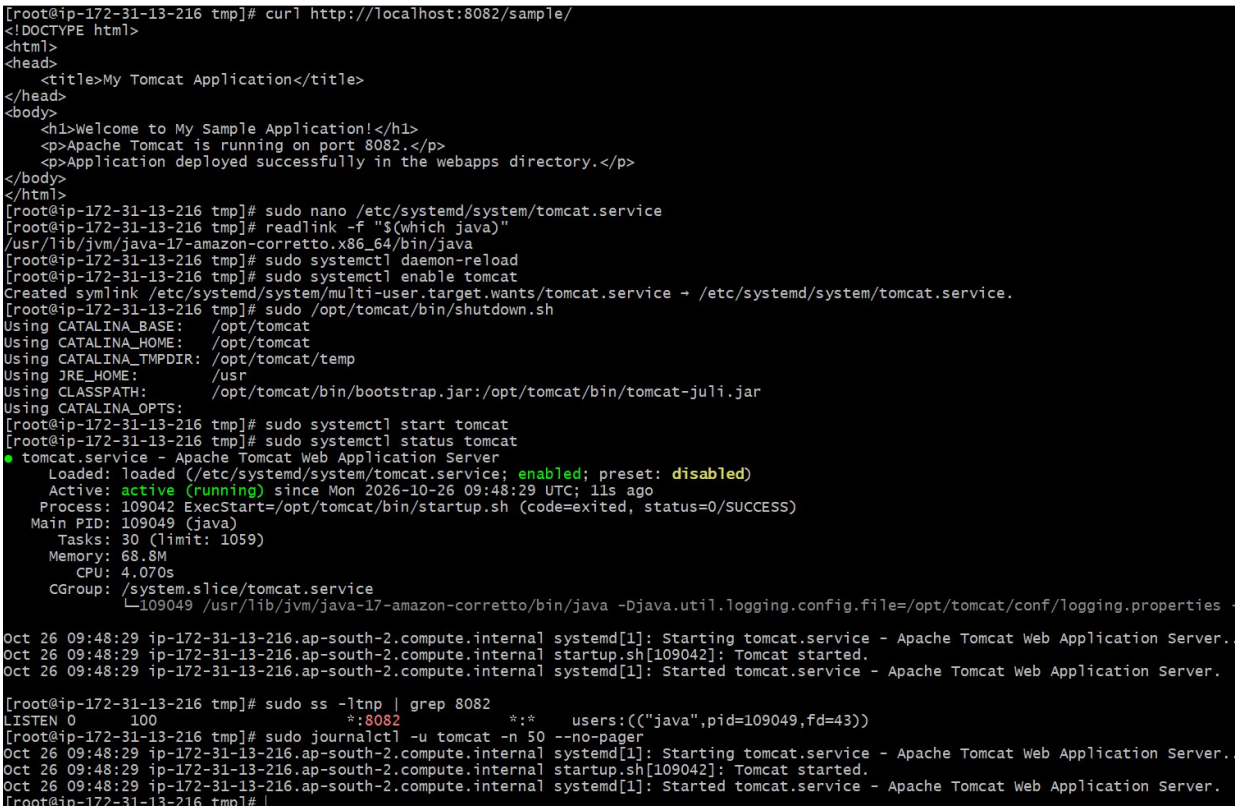
Check status:

```
sudo systemctl status tomcat
```

View logs if Tomcat fails to start:

```
sudo journalctl -u tomcat --no-pager -n 50
```

Important: If Tomcat was already started manually using `startup.sh`, stop that instance before starting it through systemd to avoid conflicts.



8, Configure HA Proxy server

Step 1: Install HAProxy

Run these commands on your Amazon Linux EC2 instance:

```
sudo dnf update -y
sudo dnf install haproxy -y
```

Check the installed version:

```
haproxy -v
```

Step 2: Configure HAProxy

Back up the existing configuration file:

```
sudo cp /etc/haproxy/haproxy.cfg /etc/haproxy/haproxy.cfg.bak
```

Open the configuration file:

```
sudo nano /etc/haproxy/haproxy.cfg
```

Keep the existing `global` and `defaults` sections. Add the following configuration at the end of the file, making sure you do not already have conflicting `frontend` or `backend` sections with these names.

```
frontend tomcat_frontend
    bind *:80
    default_backend tomcat_backend

backend tomcat_backend
    balance roundrobin
    server tomcat1 127.0.0.1:8082 check
```

Save the file using `Ctrl + O`, press `Enter`, and exit using `Ctrl + X`.

Step 3: Test the HAProxy Configuration

Run:

```
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
```

Expected output:

```
Configuration file is valid
```

The exact wording may vary by HAProxy version, but the command should report that the configuration is valid.

Step 4: Start HAProxy

Enable HAProxy to start automatically after a reboot and start it now:

```
sudo systemctl enable --now haproxy
```

Check the service status:

```
sudo systemctl status haproxy
```

Confirm that the service shows `active (running)`.

Step 5: Verify Both Ports

Check whether HAProxy is listening on port 80:

```
sudo ss -ltnp | grep ':80'
```

Check whether Tomcat is listening on port 8082:

```
sudo ss -ltnp | grep ':8082'
```

Test the backend directly:

```
curl http://localhost:8082
```

Then test HAProxy:

```
curl http://localhost
```

Both commands should return the Tomcat welcome page or other content served by Tomcat.

Step 6: Configure the AWS Security Group

Open AWS Console → EC2 → Instances → select your instance → Security → associated security group → Edit inbound rules.

Allow these ports as needed:

Port	Purpose
80	HAProxy public access
8082	Direct Tomcat testing, if needed

For normal reverse-proxy access, port 80 should be allowed from your intended client IPs. You generally do not need to expose port 8082 publicly if HAProxy and Tomcat are running on the same instance.

Step 7: Access the Website

Open your browser and enter:

```
http://YOUR-EC2-PUBLIC-IP
```

If you deployed the sample Tomcat application earlier, access it through HAProxy using:

```
http://YOUR-EC2-PUBLIC-IP/sample/
```

HAProxy should forward the request to Tomcat on port 8082.

Welcome to My Sample Application!

Tomcat is running successfully on port 8082

Sample application deployed successfully.